# 9.1 Issues in Sorting on Parallel Computers

Parallelizing a sequential sorting algorithm involves distributing the elements to be sorted onto the available processes. This process raises a number of issues that we must address in order to make the presentation of parallel sorting algorithms clearer.

## 9.1.1 Where the Input and Output Sequences are Stored

In sequential sorting algorithms, the input and the sorted sequences are stored in the process's memory. However, in parallel sorting there are two places where these sequences can reside. They may be stored on only one of the processes, or they may be distributed among the processes. The latter approach is particularly useful if sorting is an intermediate step in another algorithm. In this chapter, we assume that the input and sorted sequences are distributed among the processes.

Consider the precise distribution of the sorted output sequence among the processes. A general method of distribution is to enumerate the processes and use this enumeration to specify a global ordering for the sorted sequence. In other words, the sequence will be sorted with respect to this process enumeration. For instance, if $P_i$ comes before $P_j$ in the enumeration, all the elements stored in $P_i$ will be smaller than those stored in $P_j$ . We can enumerate the processes in many ways. For certain parallel algorithms and interconnection networks, some enumerations lead to more efficient parallel formulations than others.

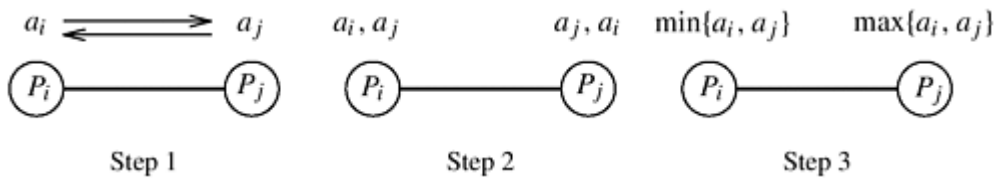
## 9.1.2 How Comparisons are Performed

A sequential sorting algorithm can easily perform a compare-exchange on two elements because they are stored locally in the process's memory. In parallel sorting algorithms, this step is not so easy. If the elements reside on the same process, the comparison can be done easily. But if the elements reside on different processes, the situation becomes more complicated.

### One Element Per Process

Consider the case in which each process holds only one element of the sequence to be sorted. At some point in the execution of the algorithm, a pair of processes ($P_i$, $P_j$) may need to compare their elements, $a_i$ and $a_j$. After the comparison, $P_i$ will hold the smaller and $P_j$ the larger of $\{a_i, a_j\}$. We can perform comparison by having both processes send their elements to each other. Each process compares the received element with its own and retains the appropriate element. In our example, $P_i$ will keep the smaller and $P_j$ will keep the larger of $\{a_i, a_j\}$. As in the sequential case, we refer to this operation as *compare-exchange*. As Figure 9.1 illustrates, each compare-exchange operation requires one comparison step and one communication step.

Figure 9.1. A parallel compare-exchange operation. Processes $P_i$ and $P_j$ send their elements to each other. Process $P_i$ keeps $\min\{a_i, a_j\}$, and $P_j$ keeps $\max\{a_i , a_j\}$.



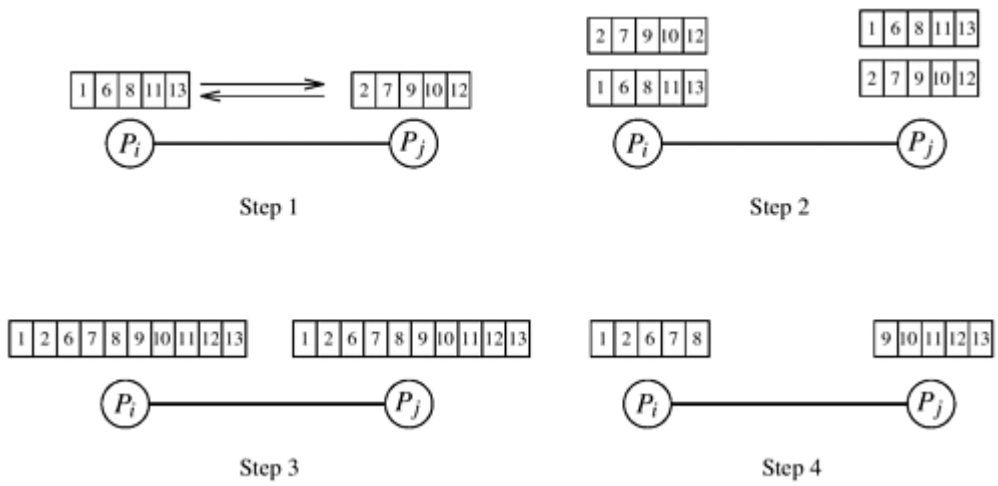

If we assume that processes $P_i$ and $P_j$ are neighbors, and the communication channels are bidirectional, then the communication cost of a compare-exchange step is $(t_s + t_w)$, where $t_s$ and $t_w$ are message-startup time and per-word transfer time, respectively. In commercially available message-passing computers, $t_s$ is significantly larger than $t_w$, so the communication time is dominated by $t_s$. Note that in today's parallel computers it takes more time to send an element from one process to another than it takes to compare the elements. Consequently, any parallel sorting formulation that uses as many processes as elements to be sorted will deliver very poor performance because the overall parallel run time will be dominated by interprocess communication.

## More than One Element Per Process

A general-purpose parallel sorting algorithm must be able to sort a large sequence with a relatively small number of processes. Let $p$ be the number of processes $P_0, P_1, \ldots, P_{p-1}$, and let $n$ be the number of elements to be sorted. Each process is assigned a block of $n/p$ elements, and all the processes cooperate to sort the sequence. Let $A_0, A_1, \ldots A_{p-1}$ be the blocks assigned to processes $P_0, P_1, \ldots P_{p-1}$, respectively. We say that $A_i \leq A_j$ if every element of $A_i$ is less than or equal to every element in $A_j$. When the sorting algorithm finishes, each process $P_i$ holds a set $A'_i$ such that $A'_i \leq A'_j$ for $i \leq j$, and $\bigcup_{i=0}^{p-1} A_i = \bigcup_{i=0}^{p-1} A'_i$.

As in the one-element-per-process case, two processes $P_i$ and $P_j$ may have to redistribute their blocks of $n/p$ elements so that one of them will get the smaller $n/p$ elements and the other will get the larger $n/p$ elements. Let $A_i$ and $A_j$ be the blocks stored in processes $P_i$ and $P_j$. If the block of $n/p$ elements at each process is already sorted, the redistribution can be done efficiently as follows. Each process sends its block to the other process. Now, each process merges the two sorted blocks and retains only the appropriate half of the merged block. We refer to this operation of comparing and splitting two sorted blocks as *compare-split*. The compare-split operation is illustrated in Figure 9.2.

Figure 9.2. A compare-split operation. Each process sends its block of size $n/p$ to the other process. Each process merges the received block with its own block and retains only the appropriate half of the merged block. In this example, process $P_i$ retains the smaller elements and process $P_j$ retains the larger elements.




If we assume that processes $P_i$ and $P_j$ are neighbors and that the communication channels are bidirectional, then the communication cost of a compare-split operation is $(t_s + t_w n/p)$. As the block size increases, the significance of $t_s$ decreases, and for sufficiently large blocks it can be ignored. Thus, the time required to merge two sorted blocks of $n/p$ elements is $\Theta(n/p)$.

# 9.2 Sorting Networks

In the quest for fast sorting methods, a number of networks have been designed that sort $n$ elements in time significantly smaller than $\Theta(n \log n)$. These sorting networks are based on a comparison network model, in which many comparison operations are performed simultaneously.

The key component of these networks is a *comparator*. A comparator is a device with two inputs $x$ and $y$ and two outputs $x'$ and $y'$. For an *increasing comparator*, $x' = \min\{x, y\}$ and $y' = \max\{x, y\}$; for a *decreasing comparator* $x' = \max\{x, y\}$ and $y' = \min\{x, y\}$. Figure 9.3 gives the schematic representation of the two types of comparators. As the two elements enter the input wires of the comparator, they are compared and, if necessary, exchanged before they go to the output wires. We denote an increasing comparator by $\oplus$ and a decreasing comparator by $\ominus$. A sorting network is usually made up of a series of columns, and each column contains a number of comparators connected in parallel. Each column of comparators performs a permutation, and the output obtained from the final column is sorted in increasing or decreasing order. Figure 9.4 illustrates a typical sorting network. The *depth* of a network is the number of columns it contains. Since the speed of a comparator is a technology-dependent constant, the speed of the network is proportional to its depth.

Figure 9.3. A schematic representation of comparators: (a) an increasing comparator, and (b) a decreasing comparator.

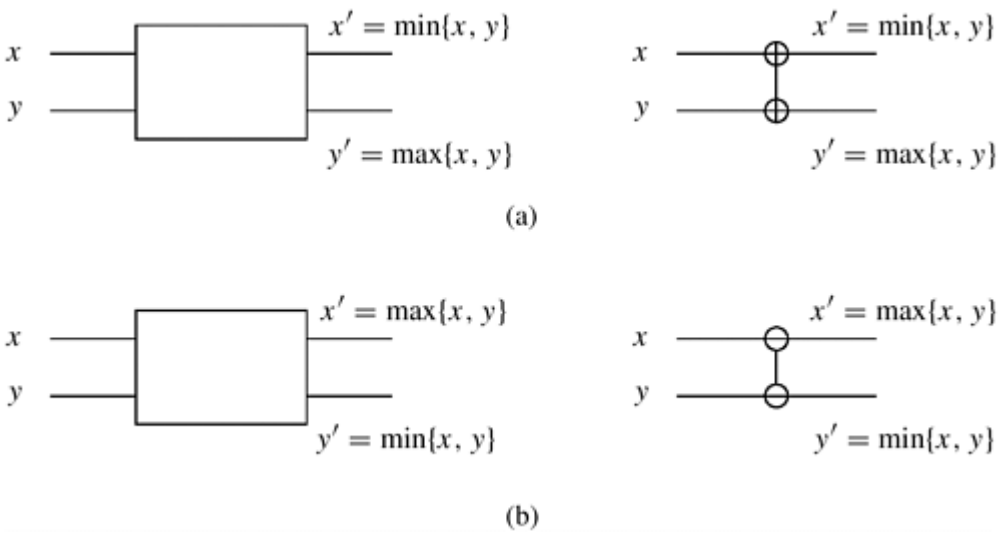
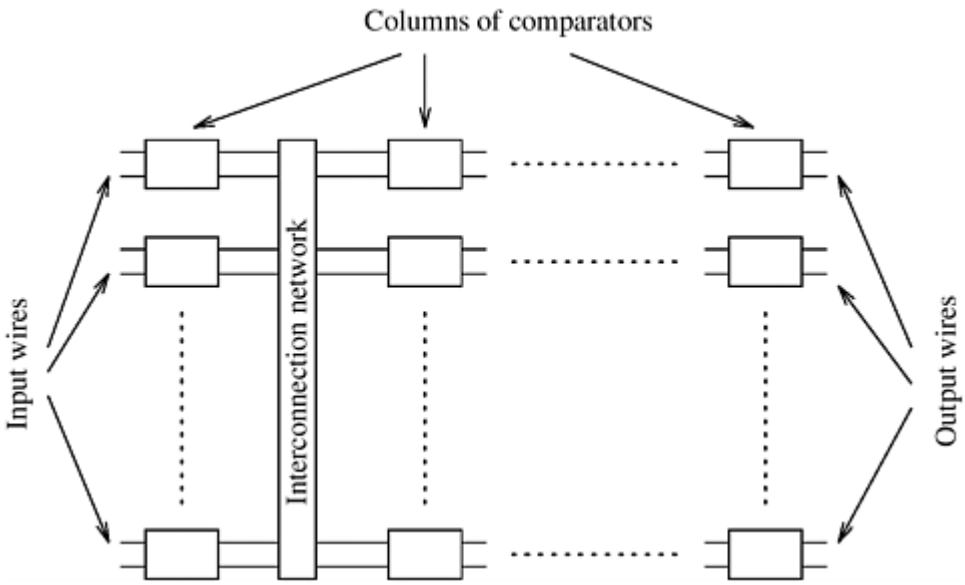



Figure 9.4. A typical sorting network. Every sorting network is made up of a series of columns, and each column contains a number of comparators connected in parallel.




We can convert any sorting network into a sequential sorting algorithm by emulating the comparators in software and performing the comparisons of each column sequentially. The comparator is emulated by a compare-exchange operation, where $x$ and $y$ are compared and, if necessary, exchanged.

The following section describes a sorting network that sorts $n$ elements in $\Theta(\log^2 n)$ time. To simplify the presentation, we assume that $n$ is a power of two.

## 9.2.1 Bitonic Sort

A bitonic sorting network sorts $n$ elements in $\Theta(\log^2 n)$ time. The key operation of the bitonic sorting network is the rearrangement of a bitonic sequence into a sorted sequence. A *bitonic sequence* is a sequence of elements $<a_0, a_1, \ldots, a_{n-1}>$ with the property that either (1) there exists an index $i$, $0 \leq i \leq n - 1$, such that $<a_0, \ldots, a_i>$ is monotonically increasing and $<a_{i+1}, \ldots, a_{n-1}>$ is monotonically decreasing, or (2) there exists a cyclic shift of indices so that (1) is satisfied. For example, $<1, 2, 4, 7, 6, 0>$ is a bitonic sequence, because it first increases and then decreases. Similarly, $<8, 9, 2, 1, 0, 4>$ is another bitonic sequence, because it is a cyclic shift of $<0, 4, 8, 9, 2, 1>$.

We present a method to rearrange a bitonic sequence to obtain a monotonically increasing sequence. Let $s = <a_0, a_1, \ldots, a_{n-1}>$ be a bitonic sequence such that $a_0 \leq a_1 \leq \ldots \leq a_{n/2-1}$ and $a_{n/2} \geq a_{n/2+1} \geq \ldots \geq a_{n-1}$. Consider the following subsequences of $s$:

### Equation 9.1

$$
\begin{aligned}
s_1 &= \langle \min\{a_0, a_{n/2}\}, \min\{a_1, a_{n/2+1}\}, \ldots, \min\{a_{n/2-1}, a_{n-1}\} \rangle \\
s_2 &= \langle \max\{a_0, a_{n/2}\}, \max\{a_1, a_{n/2+1}\}, \ldots, \max\{a_{n/2-1}, a_{n-1}\} \rangle
\end{aligned}
$$

In sequence $s_1$, there is an element $b_i = \min\{a_i, a_{n/2+i}\}$ such that all the elements before $b_i$ are from the increasing part of the original sequence and all the elements after $b_i$ are from the decreasing part. Also, in sequence $s_2$, the element $b'_i = \max\{a_i, a_{n/2+i}\}$ is such that all the

elements before $b'_i$ are from the decreasing part of the original sequence and all the elements after $b'_i$ are from the increasing part. Thus, the sequences $s_1$ and $s_2$ are bitonic sequences. Furthermore, every element of the first sequence is smaller than every element of the second sequence. The reason is that $b_i$ is greater than or equal to all elements of $s_1$, $b'_i$ is less than or equal to all elements of $s_2$, and $b'_i$ is greater than or equal to $b_i$. Thus, we have reduced the initial problem of rearranging a bitonic sequence of size $n$ to that of rearranging two smaller bitonic sequences and concatenating the results. We refer to the operation of splitting a bitonic sequence of size $n$ into the two bitonic sequences defined by Equation 9.1 as a *bitonic split*. Although in obtaining $s_1$ and $s_2$ we assumed that the original sequence had increasing and decreasing sequences of the same length, the bitonic split operation also holds for any bitonic sequence (Problem 9.3).

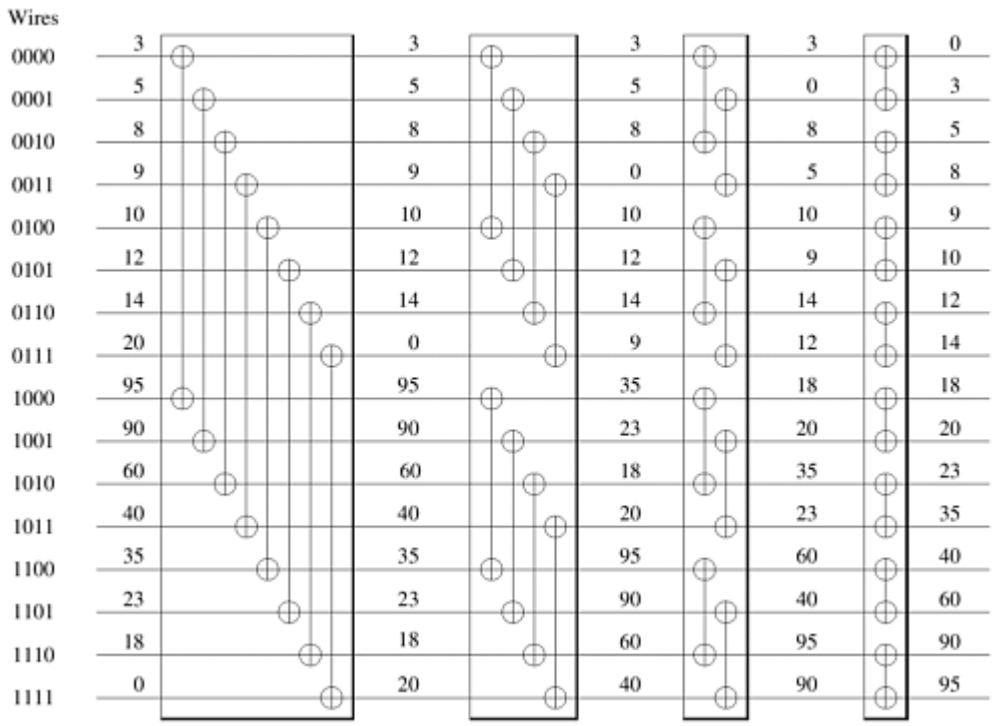
We can recursively obtain shorter bitonic sequences using Equation 9.1 for each of the bitonic subsequences until we obtain subsequences of size one. At that point, the output is sorted in monotonically increasing order. Since after each bitonic split operation the size of the problem is halved, the number of splits required to rearrange the bitonic sequence into a sorted sequence is log $n$. The procedure of sorting a bitonic sequence using bitonic splits is called *bitonic merge*. The recursive bitonic merge procedure is illustrated in Figure 9.5.

Figure 9.5. Merging a 16-element bitonic sequence through a series of log 16 bitonic splits.

| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Original sequence | 3 | 5 | 8 | 9 | 10 | 12 | 14 | 20 | 95 | 90 | 60 | 40 | 35 | 23 | 18 | 0 |
| 1st Split | 3 | 5 | 8 | 9 | 10 | 12 | 14 | 0 | 95 | 90 | 60 | 40 | 35 | 23 | 18 | 20 |
| 2nd Split | 3 | 5 | 8 | 0 | 10 | 12 | 14 | 9 | 35 | 23 | 18 | 20 | 95 | 90 | 60 | 40 |
| 3rd Split | 3 | 0 | 8 | 5 | 10 | 9 | 14 | 12 | 18 | 20 | 35 | 23 | 60 | 40 | 95 | 90 |
| 4th Split | 0 | 3 | 5 | 8 | 9 | 10 | 12 | 14 | 18 | 20 | 23 | 35 | 40 | 60 | 90 | 95 |

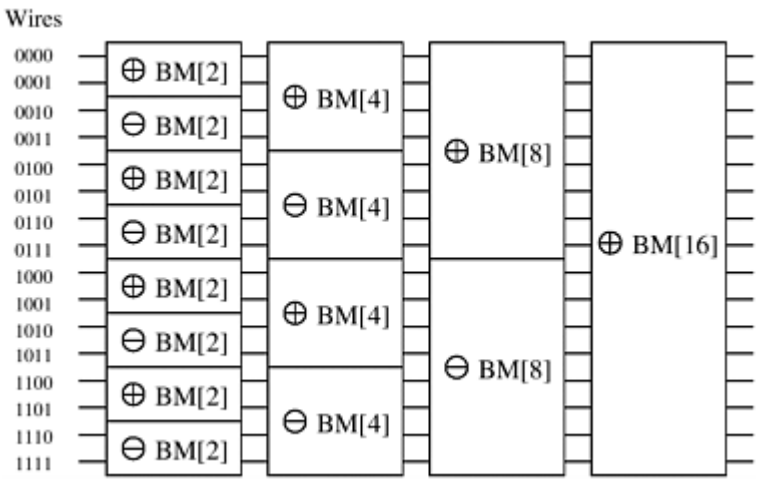
We now have a method for merging a bitonic sequence into a sorted sequence. This method is easy to implement on a network of comparators. This network of comparators, known as a *bitonic merging network*, it is illustrated in Figure 9.6. The network contains log $n$ columns. Each column contains $n/2$ comparators and performs one step of the bitonic merge. This network takes as input the bitonic sequence and outputs the sequence in sorted order. We denote a bitonic merging network with $n$ inputs by $\oplus$BM[n]. If we replace the $\oplus$ comparators in Figure 9.6 by $\ominus$ comparators, the input will be sorted in monotonically decreasing order; such a network is denoted by $\ominus$BM[n].

Figure 9.6. A bitonic merging network for $n$ = 16. The input wires are numbered 0, 1 ..., $n$ - 1, and the binary representation of these numbers is shown. Each column of comparators is drawn separately; the entire figure represents a $\oplus$BM[16] bitonic merging network. The network takes a bitonic sequence and outputs it in sorted order.




Armed with the bitonic merging network, consider the task of sorting *n* unordered elements. This is done by repeatedly merging bitonic sequences of increasing length, as illustrated in Figure 9.7.

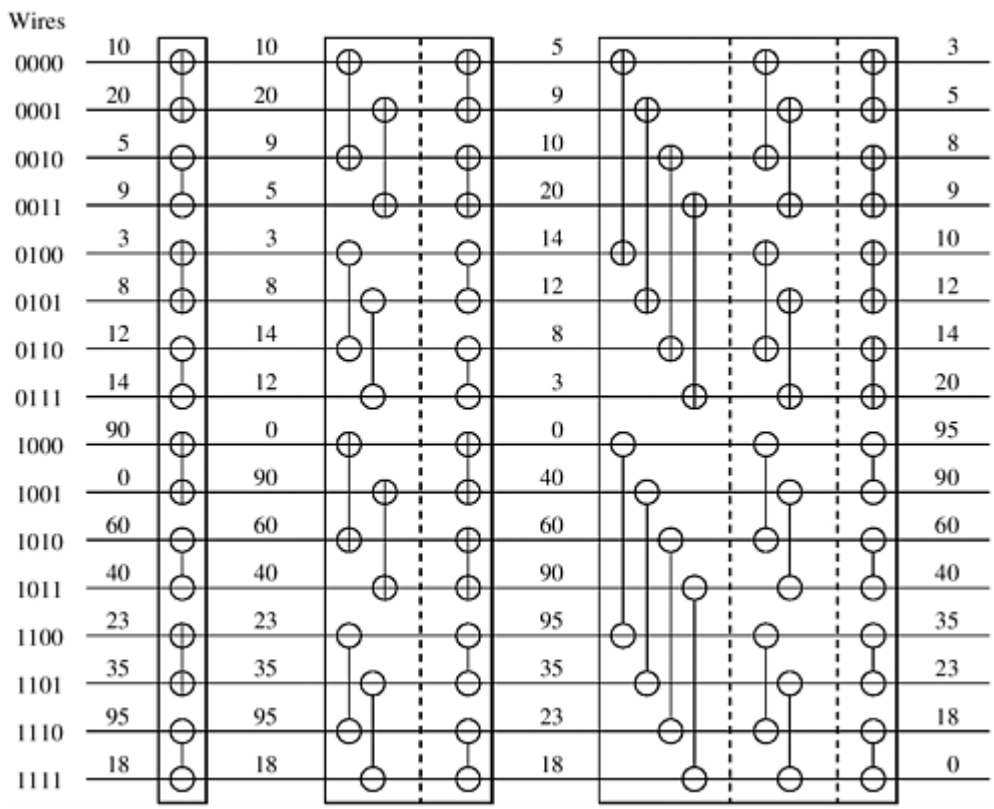
Figure 9.7. A schematic representation of a network that converts an input sequence into a bitonic sequence. In this example, $\oplus$BM[k] and $\ominus$BM[k] denote bitonic merging networks of input size *k* that use $\oplus$ and $\ominus$ comparators, respectively. The last merging network ($\oplus$BM[16]) sorts the input. In this example, *n* = 16.




Let us now see how this method works. A sequence of two elements *x* and *y* forms a bitonic sequence, since either $x \leq y$, in which case the bitonic sequence has *x* and *y* in the increasing part and no elements in the decreasing part, or $x \geq y$, in which case the bitonic sequence has *x* and *y* in the decreasing part and no elements in the increasing part. Hence, any unsorted sequence of elements is a concatenation of bitonic sequences of size two. Each stage of the

network shown in Figure 9.7 merges adjacent bitonic sequences in increasing and decreasing order. According to the definition of a bitonic sequence, the sequence obtained by concatenating the increasing and decreasing sequences is bitonic. Hence, the output of each stage in the network in Figure 9.7 is a concatenation of bitonic sequences that are twice as long as those at the input. By merging larger and larger bitonic sequences, we eventually obtain a bitonic sequence of size *n*. Merging this sequence sorts the input. We refer to the algorithm embodied in this method as *bitonic sort* and the network as a *bitonic sorting network*. The first three stages of the network in Figure 9.7 are shown explicitly in Figure 9.8. The last stage of Figure 9.7 is shown explicitly in Figure 9.6.

Figure 9.8. The comparator network that transforms an input sequence of 16 unordered numbers into a bitonic sequence. In contrast to Figure 9.6, the columns of comparators in each bitonic merging network are drawn in a single box, separated by a dashed line.




The last stage of an *n*-element bitonic sorting network contains a bitonic merging network with *n* inputs. This has a depth of log *n*. The other stages perform a complete sort of *n*/2 elements. Hence, the depth, *d*(*n*), of the network in Figure 9.7 is given by the following recurrence relation:

## Equation 9.2

$$d(n) = d(n/2) + \log n$$

Solving Equation 9.2, we obtain $d(n) = \sum_{i=1}^{\log n} i = (\log^2 n + \log n)/2 = \Theta(\log^2 n)$. This network can be implemented on a serial computer, yielding a $\Theta(n \log^2 n)$ sorting algorithm. The bitonic

sorting network can also be adapted and used as a sorting algorithm for parallel computers. In the next section, we describe how this can be done for hypercube-and mesh-connected parallel computers.

## 9.2.2 Mapping Bitonic Sort to a Hypercube and a Mesh

In this section we discuss how the bitonic sort algorithm can be mapped on general-purpose parallel computers. One of the key aspects of the bitonic algorithm is that it is communication intensive, and a proper mapping of the algorithm must take into account the topology of the underlying interconnection network. For this reason, we discuss how the bitonic sort algorithm can be mapped onto the interconnection network of a hypercube- and mesh-connected parallel computers.

The bitonic sorting network for sorting $n$ elements contains log $n$ stages, and stage $i$ consists of $i$ columns of $n/2$ comparators. As Figures 9.6 and 9.8 show, each column of comparators performs compare-exchange operations on $n$ wires. On a parallel computer, the compare-exchange function is performed by a pair of processes.

### One Element Per Process

In this mapping, each of the $n$ processes contains one element of the input sequence. Graphically, each wire of the bitonic sorting network represents a distinct process. During each step of the algorithm, the compare-exchange operations performed by a column of comparators are performed by $n/2$ pairs of processes. One important question is how to map processes to wires in order to minimize the distance that the elements travel during a compare-exchange operation. If the mapping is poor, the elements travel a long distance before they can be compared, which will degrade performance. Ideally, wires that perform a compare-exchange should be mapped onto neighboring processes. Then the parallel formulation of bitonic sort will have the best possible performance over all the formulations that require $n$ processes.

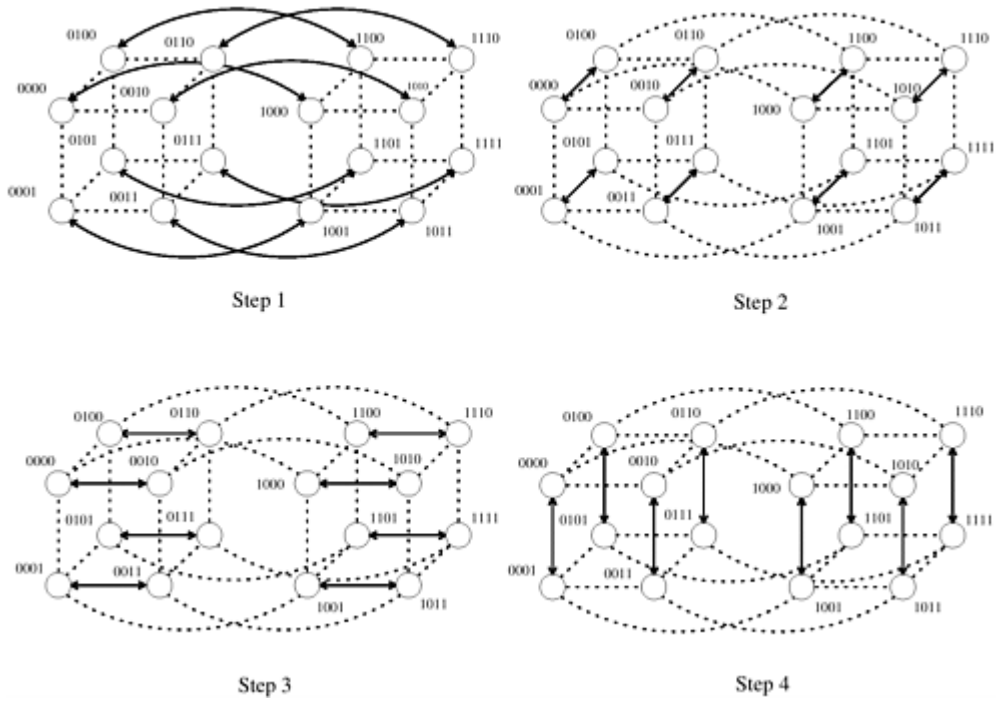
To obtain a good mapping, we must further investigate the way that input wires are paired during each stage of bitonic sort. Consider Figures 9.6 and 9.8, which show the full bitonic sorting network for $n = 16$. In each of the $(1 + \log 16)(\log 16)/2 = 10$ comparator columns, certain wires compare-exchange their elements. Focus on the binary representation of the wire labels. In any step, the compare-exchange operation is performed between two wires only if their labels differ in exactly one bit. During each of the four stages, wires whose labels differ in the least-significant bit perform a compare-exchange in the last step of each stage. During the last three stages, wires whose labels differ in the second-least-significant bit perform a compare-exchange in the second-to-last step of each stage. In general, wires whose labels differ in the $i^{th}$ least-significant bit perform a compare-exchange $(\log n - i + 1)$ times. This observation helps us efficiently map wires onto processes by mapping wires that perform compare-exchange operations more frequently to processes that are close to each other.

Hypercube Mapping wires onto the processes of a hypercube-connected parallel computer is straightforward. Compare-exchange operations take place between wires whose labels differ in only one bit. In a hypercube, processes whose labels differ in only one bit are neighbors (Section 2.4.3). Thus, an optimal mapping of input wires to hypercube processes is the one that maps an input wire with label $l$ to a process with label $l$ where $l = 0, 1, \ldots, n - 1$.

Consider how processes are paired for their compare-exchange steps in a $d$-dimensional hypercube (that is, $p = 2^d$). In the final stage of bitonic sort, the input has been converted into a bitonic sequence. During the first step of this stage, processes that differ only in the $d^{th}$ bit of the binary representation of their labels (that is, the most significant bit) compare-exchange

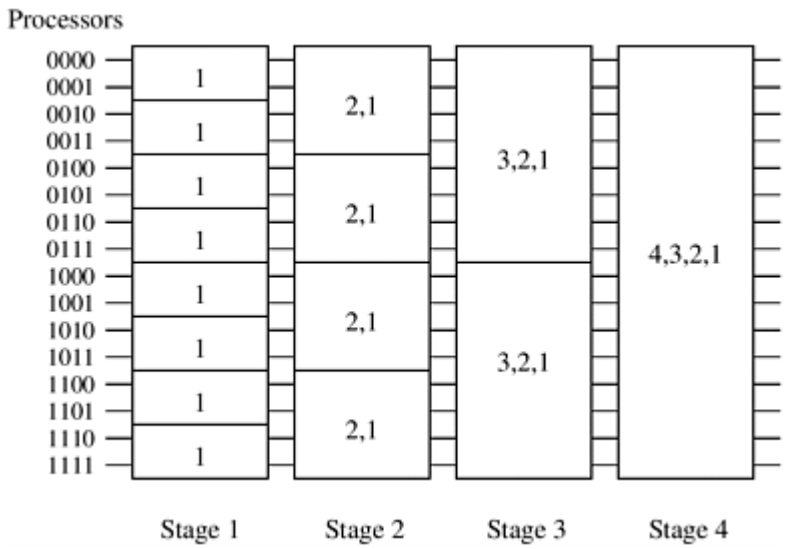
their elements. Thus, the compare-exchange operation takes place between processes along the $d^{th}$ dimension. Similarly, during the second step of the algorithm, the compare-exchange operation takes place among the processes along the $(d - 1)^{th}$ dimension. In general, during the $i^{th}$ step of the final stage, processes communicate along the $(d - (i - 1))^{th}$ dimension. Figure 9.9 illustrates the communication during the last stage of the bitonic sort algorithm.

Figure 9.9. Communication during the last stage of bitonic sort. Each wire is mapped to a hypercube process; each connection represents a compare-exchange between processes.




A bitonic merge of sequences of size $2^k$ can be performed on a $k$-dimensional subcube, with each such sequence assigned to a different subcube (Problem 9.5). Furthermore, during the $i^{th}$ step of this bitonic merge, the processes that compare their elements are neighbors along the $(k - (i - 1))^{th}$ dimension. Figure 9.10 is a modification of Figure 9.7, showing the communication characteristics of the bitonic sort algorithm on a hypercube.

Figure 9.10. Communication characteristics of bitonic sort on a hypercube. During each stage of the algorithm, processes communicate along the dimensions shown.




The bitonic sort algorithm for a hypercube is shown in Algorithm 9.1. The algorithm relies on the functions *comp_exchange_max(i)* and *comp_exchange_min(i)*. These functions compare the local element with the element on the nearest process along the $i^{th}$ dimension and retain either the minimum or the maximum of the two elements. Problem 9.6 explores the correctness of Algorithm 9.1.

### Algorithm 9.1 Parallel formulation of bitonic sort on a hypercube with $n = 2^d$ processes. In this algorithm, *label* is the process's label and *d* is the dimension of the hypercube.

```
procedure BITONIC_SORT(label, d)
begin
   for i := 0 to d - 1 do
       for j := i downto 0 do
           if (i + 1)st bit of label ≠ jth bit of label then
                comp_exchange max(j);
           else
               comp_exchange min(j);
end BITONIC_SORT
```

During each step of the algorithm, every process performs a compare-exchange operation. The algorithm performs a total of $(1 + \log n)(\log n)/2$ such steps; thus, the parallel run time is

### Equation 9.3

$$T_P = \Theta(\log^2 n)$$

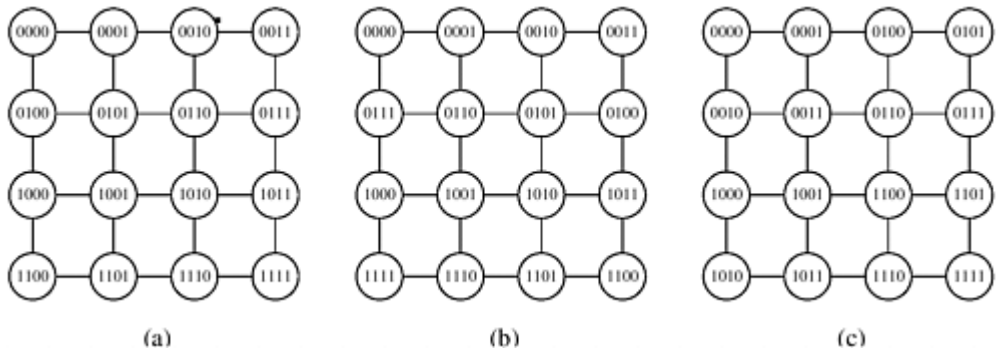
This parallel formulation of bitonic sort is cost optimal with respect to the sequential implementation of bitonic sort (that is, the process-time product is $\Theta(n \log^2 n)$), but it is not cost-optimal with respect to an optimal comparison-based sorting algorithm, which has a serial time complexity of $\Theta(n \log n)$.

Mesh Consider how the input wires of the bitonic sorting network can be mapped efficiently onto an $n$-process mesh. Unfortunately, the connectivity of a mesh is lower than that of a

hypercube, so it is impossible to map wires to processes such that each compare-exchange operation occurs only between neighboring processes. Instead, we map wires such that the most frequent compare-exchange operations occur between neighboring processes.

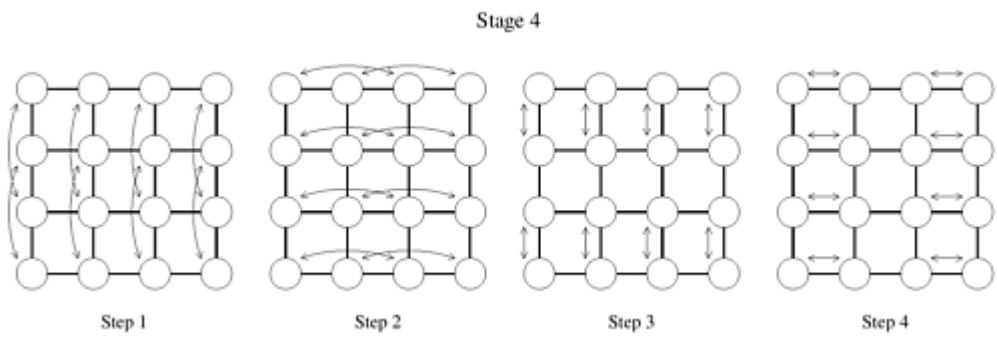
There are several ways to map the input wires onto the mesh processes. Some of these are illustrated in Figure 9.11. Each process in this figure is labeled by the wire that is mapped onto it. Of these three mappings, we concentrate on the row-major shuffled mapping, shown in Figure 9.11(c). We leave the other two mappings as exercises (Problem 9.7).

Figure 9.11. Different ways of mapping the input wires of the bitonic sorting network to a mesh of processes: (a) row-major mapping, (b) row-major snakelike mapping, and (c) row-major shuffled mapping.




The advantage of row-major shuffled mapping is that processes that perform compare-exchange operations reside on square subsections of the mesh whose size is inversely related to the frequency of compare-exchanges. For example, processes that perform compare-exchange during every stage of bitonic sort (that is, those corresponding to wires that differ in the least-significant bit) are neighbors. In general, wires that differ in the $i^{th}$ least-significant bit are mapped onto mesh processes that are $2^{\lfloor (i-1)/2 \rfloor}$ communication links away. The compare-exchange steps of the last stage of bitonic sort for the row-major shuffled mapping are shown in Figure 9.12. Note that each earlier stage will have only some of these steps.

Figure 9.12. The last stage of the bitonic sort algorithm for $n$ = 16 on a mesh, using the row-major shuffled mapping. During each step, process pairs compare-exchange their elements. Arrows indicate the pairs of processes that perform compare-exchange operations.




During the (1 + log $n$)(log $n$)/2 steps of the algorithm, processes that are a certain distance apart compare-exchange their elements. The distance between processes determines the communication overhead of the parallel formulation. The total amount of communication performed by each process is $\sum_{i=1}^{\log n} \sum_{j=1}^{i} 2^{\lfloor (j-1)/2 \rfloor} \approx 7\sqrt{n}$, which is $\Theta(\sqrt{n})$ (Problem 9.7).

During each step of the algorithm, each process performs at most one comparison; thus, the total computation performed by each process is $\Theta(\log^2 n)$. This yields a parallel run time of

$$T_P = \overbrace{\Theta(\log^2 n)}^{\text{comparisons}} + \overbrace{\Theta(\sqrt{n}).}^{\text{communication}}$$

This is not a cost-optimal formulation, because the process-time product is $\Theta(n^{1.5})$, but the sequential complexity of sorting is $\Theta(n \log n)$. Although the parallel formulation for a hypercube was optimal with respect to the sequential complexity of bitonic sort, the formulation for mesh is not. Can we do any better? No. When sorting $n$ elements, one per mesh process, for certain inputs the element stored in the process at the upper-left corner will end up in the process at the lower-right corner. For this to happen, this element must travel along $2\sqrt{n} - 1$ communication links before reaching its destination. Thus, the run time of sorting on a mesh is bounded by $\Omega(\sqrt{n})$. Our parallel formulation achieves this lower bound; thus, it is asymptotically optimal for the mesh architecture.

## A Block of Elements Per Process

In the parallel formulations of the bitonic sort algorithm presented so far, we assumed there were as many processes as elements to be sorted. Now we consider the case in which the number of elements to be sorted is greater than the number of processes.

Let $p$ be the number of processes and $n$ be the number of elements to be sorted, such that $p < n$. Each process is assigned a block of $n/p$ elements and cooperates with the other processes to sort them. One way to obtain a parallel formulation with our new setup is to think of each process as consisting of $n/p$ smaller processes. In other words, imagine emulating $n/p$ processes by using a single process. The run time of this formulation will be greater by a factor of $n/p$ because each process is doing the work of $n/p$ processes. This virtual process approach (Section 5.3) leads to a poor parallel implementation of bitonic sort. To see this, consider the case of a hypercube with $p$ processes. Its run time will be $\Theta((n \log^2 n)/p)$, which is not cost-optimal because the process-time product is $\Theta(n \log^2 n)$.

An alternate way of dealing with blocks of elements is to use the compare-split operation presented in Section 9.1. Think of the $(n/p)$-element blocks as elements to be sorted using compare-split operations. The problem of sorting the $p$ blocks is identical to that of performing a bitonic sort on the $p$ blocks using compare-split operations instead of compare-exchange operations (Problem 9.8). Since the total number of blocks is $p$, the bitonic sort algorithm has a total of $(1 + \log p)(\log p)/2$ steps. Because compare-split operations preserve the initial sorted order of the elements in each block, at the end of these steps the $n$ elements will be sorted. The main difference between this formulation and the one that uses virtual processes is that the $n/p$ elements assigned to each process are initially sorted locally, using a fast sequential sorting algorithm. This initial local sort makes the new formulation more efficient and cost-optimal.

Hypercube The block-based algorithm for a hypercube with $p$ processes is similar to the one-element-per-process case, but now we have $p$ blocks of size $n/p$, instead of $p$ elements. Furthermore, the compare-exchange operations are replaced by compare-split operations, each taking $\Theta(n/p)$ computation time and $\Theta(n/p)$ communication time. Initially the processes sort their $n/p$ elements (using merge sort) in time $\Theta((n/p) \log(n/p))$ and then perform $\Theta(\log^2 p)$ compare-split steps. The parallel run time of this formulation is

$$T_P = \overbrace{\Theta\left(\frac{n}{p}\log\frac{n}{p}\right)}^{\text{local sort}} + \overbrace{\Theta\left(\frac{n}{p}\log^2 p\right)}^{\text{comparisons}} + \overbrace{\Theta\left(\frac{n}{p}\log^2 p\right)}^{\text{communication}}.$$

Because the sequential complexity of the best sorting algorithm is $\Theta(n \log n)$, the speedup and efficiency are as follows:

## Equation 9.4

$$\begin{aligned} S &= \frac{\Theta(n\log n)}{\Theta((n/p)\log(n/p)) + \Theta((n/p)\log^2 p)} \\ E &= \frac{1}{1 - \Theta((\log p)/(\log n)) + \Theta((\log^2 p)/(\log n))} \end{aligned}$$

From Equation 9.4, for a cost-optimal formulation $(\log^2 p)/(\log n) = O(1)$. Thus, this algorithm can efficiently use up to $p = \Theta(2^{\sqrt{\log n}})$ processes. Also from Equation 9.4, the isoefficiency function due to both communication and extra work is $\Theta(p^{\log p} \log^2 p)$, which is worse than any polynomial isoefficiency function for sufficiently large $p$. Hence, this parallel formulation of bitonic sort has poor scalability.

Mesh The block-based mesh formulation is also similar to the one-element-per-process case. The parallel run time of this formulation is as follows:

$$T_P = \overbrace{\Theta\left(\frac{n}{p}\log\frac{n}{p}\right)}^{\text{local sort}} + \overbrace{\Theta\left(\frac{n}{p}\log^2 p\right)}^{\text{comparisons}} + \overbrace{\Theta\left(\frac{n}{\sqrt{p}}\right)}^{\text{communication}}$$

Note that comparing the communication overhead of this mesh-based parallel bitonic sort $(O(n/\sqrt{p}))$ to the communication overhead of the hypercube-based formulation $(O((n \log^2 p)/p))$, we can see that it is higher by a factor of $O(\sqrt{p}/log^2 p)$. This factor is smaller than the $O(\sqrt{p})$ difference in the bisection bandwidth of these architectures. This illustrates that a proper mapping of the bitonic sort on the underlying mesh can achieve better performance than that achieved by simply mapping the hypercube algorithm on the mesh.

The speedup and efficiency are as follows:

## Equation 9.5

$$\begin{aligned} S &= \frac{\Theta(n\log n)}{\Theta((n/p)\log(n/p)) + \Theta((n/p)\log^2 p) + \Theta(n/\sqrt{p})} \\ E &= \frac{1}{1 - \Theta((\log p)/(\log n)) + \Theta((\log^2 p)/(\log n)) + \Theta(\sqrt{p}/\log n)} \end{aligned}$$

Table 9.1. The performance of parallel formulations of bitonic sort for $n$ elements on $p$ processes.

| Architecture | Maximum Number of Processes for $E$ = Q(1) | Corresponding Parallel Run Time | Isoefficiency Function |
|---|---|---|---|
| Hypercube | $\Theta(2^{\sqrt{\log n}})$ | $\Theta(n/(2^{\sqrt{\log n}})\log n)$ | $\Theta(p^{\log p}\log^2 p)$ |
| Mesh | $\Theta(\log^2 n)$ | $\Theta(n/\log n)$ | $\Theta(2^{\sqrt{p}}\sqrt{p})$ |
| Ring | $\Theta(\log n)$ | $\Theta(n)$ | $\Theta(2^p p)$ |

From Equation 9.5, for a cost-optimal formulation $\sqrt{p}/\log n = O(1)$. Thus, this formulation can efficiently use up to $p = \Theta(\log^2 n)$ processes. Also from Equation 9.5, the isoefficiency function $\Theta(2^{\sqrt{p}}\sqrt{p})$. The isoefficiency function of this formulation is exponential, and thus is even worse than that for the hypercube.

From the analysis for hypercube and mesh, we see that parallel formulations of bitonic sort are neither very efficient nor very scalable. This is primarily because the sequential algorithm is suboptimal. Good speedups are possible on a large number of processes only if the number of elements to be sorted is very large. In that case, the efficiency of the internal sorting outweighs the inefficiency of the bitonic sort. Table 9.1 summarizes the performance of bitonic sort on hypercube-, mesh-, and ring-connected parallel computer.